a/ = 5

System.out.println(a); = 4

a% = 4

System.out.println(a); = 0

class: 4

15/04/2025

* Relational Operators

BODMAS Rule:

B - Bracket - ( )

O - Of

D - Division

M - Multiplication

A - Addition

S - Substraction.

Ex: 1) System.out.println(num1 + num2 * num1)

num1 = 5
num2 = 10

// 5 + 10 * 5 = 5 + 50 = 55

output: 55.

* Java follows BODMAS Rule.

2) System.out.println(num1 - num1 * num2 + (num1 + num2));

num1 = 10
num2 = 5

// (10 - 10 * 5 + (10 + 5))

(10 - 50 + 15) = -25

output: -25

3. Relational Operators: [used to compare variables].

→ a) Equals — (==) output: boolean type i.e, true/false.

→ b) Greaterthan - (>)

(a > b)

→ c) Less than - (<)

(a < b)

→ d) Greater than equal to - (>=)

(a >= b)

→ e) Less than equal to - (<=)

(a <= b)

f) Not equal to — (!=)

(a != b)

Ex:

```
package project 3

public class Relational Operators {
    public static void main (String [ ] args){
        int a = 10 ; ⇒ (variable declaration)
        int b = 11 ;
        System. out. print ln ( a == b) ;
        System. out. print ln ( a > b) ;
        System. out. print ln ( a < b) ;
        System. out. print ln ( a >= b) ;
        System. out. print ln @( a <= b) ;
        System. out. print ln ( a != b) ;
    }
}
```

output: false
false
true
false
true
false.
true

[other than, we can assign variables like less than, greater than etc] ↓ (of boolean type)

* int c ; → (Declaration) of variable.
  c = 10; → (Initialisation)

* int c = 10 ; (Declaration & Initialisation of variables).
  → we have to declare one variable only once.

*

4. Unary Operators:

→ These operators perform on a single operand.

a+b [a, b called Operands, + is operator].

a) +a (5, +5)

b) -a (5, -5) - this converts into opposite sign.

c) ! (not operator) - (opposite) i.e, True makes false & Vice versa. (It works only with boolean value).

Ex: boolean a = true
!a → false

boolean b = false
!b ⇒ true.

d)

d) Increment Operator : (++)

Two types : 1. Post Increment

→ a++ (They will increase value by 1) [after crossing the operation]

2. Pre Increment

→ ++a

1. Post Increment: a++

→ It will increase value by 1 after crossing the operation.

Ex: int a = 20;

System.out.println(a); → 20

System.out.println(a++); → 20

System.out.println(a); → 21

output: 20 20 21

2. Pre-Increment Operator: ++a.

→ It will increase the value by 1 before crossing the operation.

Ex: int a = 20;

System.out.println(a); → 20

System.out.println(++a); → 21

System.out.println(a); → 21

*** System.out.println(a++); → 21

System.out.println(a); 22

System.out.println(++a); → 23

output

e) Decrement Operator : (--)

1. Post decrement (a--):

int a = 15;

syso(a); = 15

syso(a--) = 15

syso(a) = 14

2. Pre-decrement (--a);

int a = 18

syso(a) = 18

syso(--a) = 18

syso(a) = 17

Ex: package com;

public class UnaryOperators {

~~public class void s'~~

public static void main (String [ ] args) {

~~int~~

long a = 10;

long b = -20;

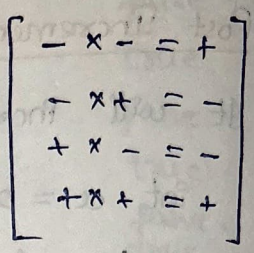
Output:

syso (+a); → +10

syso (-a); → -10

syso (-b); → +20

syso (b); → -20



boolean flag = false

syso (!flag) → true

boolean notFlag = !flag

} }

Ex: long a = 15; syso (notFlag); → true.

long b = 20;

output

syso (a); → 15

syso (a++); → 15

syso (a); → 16

syso (++a); → 17

syso (b) → 20

~~syso (b++) → 20~~

syso (--b) → 19

syso (b--) → 19

syso (b) → 18

* a=20;

~~int~~ long add = a++ + ++a

syso (add); → 20 + 22 = 42

* a=10 , b=15.

long value = --b + a++ - --a;

syso (value) ; → 14 (output).

// --b + a++ - --a.

// 14 + 10 - 10 = 14

Class: 5

* a = 5 , b = 10

```
long value2 = a++ + --b * ++a;
syso(value2);    // 5+9*7 = 5+63
```

Output: 68

16/04/2025

* Unary operators changes dyaultly + & - into int datatype. (convert it)

```
short num1 = 10
short num2 = 11
short num3 = -num1 (error)
```

So, int num3 = -num1.

* /* → multiple line comment */

* Logical Operators: Two types.
  ↳ (only works on boolean values).

* && (AND OPERATOR)
* || (OR OPERATOR)

} Both work on Booleans (i.e, true/false).

* AND OPERATOR

	Output
* T && T	→ true
* F && F	→ false
* T && F	→ false
* F && T	→ false.

OR - OPERATOR

	Output
* T || T	→ True
* T || F	→ True
* F || T	→ True
* F || F	→ False.

! (NOT OPERATOR)

* ! true → false
* ! false → true

Ex:

```
package com;
public class Logical Operators {
    public static void main(String[] args) {
        ⇒ boolean b1 = true;
        ⇒ boolean b2 = false;
        ⇒ boolean b3 = true;
          System.out.println(b1 && b2) → output: false
          syso(b1 && b3); → output: true.
          syso(b2 && b3); → output: false
        ⇒ boolean b4 = false;
          syso(b2 && b4); → output: false.
          syso(b1 || b2); → output: true.
          syso(b1 || b3); → output: true.
          syso(b2 || b3); → output: true.
          syso(b2 || b4); → output: false.
          syso(b1 && b2 && b3); → output: false.
          syso(b2 || b3 || b4); → output: true
          syso(b1 || b2 && b3); → output: true.
```

AND OPERATOR

* (&&) has highest precedence (calculate first).

```
// true || false && true
// true || false → true
syso(b1 || b2 && b3 || b4); → output: true.
// T || f && t || f
// T || f || f
// T || f → True
syso(b1 || b2 && (b3 || b4); output: True.
// T || F && (T || F);
// T || F && T
// T || F → True
```

* [:: brackets have higher precedence]

Ex:
```
int num1 = 10;
int num2 = 30;
int num3 = 10;
boolean b = (num1 < num2) && (num2 >= num3);
                 true  &&  true
Syso(b); output: true
boolean b1 = (num1 >= num2) || (num1 != num3) && (num2 < num3);
Syso(b1);          // F || F && F
output: false      // F || F → false.
```

*****

* Ternary Operator: (? :)

Condition ? execut1 : execut2

(boolean condition) * [If condition is true, (excute1) i.e, expression after question mark will execute]

* [If condition is false, expression after colon mark(:) will execute]

Ex:
```
a = 10;
b = 20;
(a<b) ? "small" : "big".   output: small.
(True)

(a>b) ? → output: big.
(false)
```

Syntax: (Condition) ? (true statement) : (false statement)

Ex:
```
package com;
public class Ternary Operator {
    psvm {
        int a = 10;
        int b = 9;
        boolean result = (a>b) ? true : false;
        Syso(result);   output: true.
        string r = (a<b) ? "a is small" : "a is big"
        Syso(r);   output: a is big
```

* [bugs → errors | logical errors] in a code.

```
int age = 19
String s = (age >= 18) ? "Can Vote" : "can't Vote";
Syso(s);   output: Can Vote.
String s = (num > 0) ? "positive" : (num == 0) ? "zero" : "Negative";
```

17/04/2025

Ex:

```
int marks = 75;
String s = (marks > 90) && (marks <= 100) ? "Grade A" :
           (marks > 70) && (marks <= 90) ? "Grade B" :
           (marks > 50) && (marks <= 70) ? "Grade C" :
           (marks > 0) && (marks <= 50) ? "Fail" : "Invalid";
System.out.println(s); → output: Grade B
```

CONDITIONAL STATEMENTS

Dynamic Inputs: ⇒ Scanner class

```
Scanner sc = new Scanner();
```

Ex:

```
package com;
import java.util.Scanner;
public class Dynamic Input {
    public static void main(String[] args) {
```

* import java.util.Scanner

```
Scanner sc = new Scanner(System.in);
sc.nextInt();
sc.nextShort();
```

⇒ (to show it will take inputs)

* Scanners used to give input directly in runtime.

```
sc.nextByte();
sc.nextLong();
sc.nextLine();
sc.next();
sc.nextFloat();
```

⇒ `int marks = sc.nextInt();`
`System.out.println(marks);` grade

* For char → use sc.next();

```
System.out.println("Enter Marks: ");
int marks = sc.nextInt();
System.out.println(marks + "marks: " + grade);
sc.close(); (optional).
```

* sc.nextLine

String name = sc.nextLine(); // will accept spaces also

String name2 = sc.next(); → // will not accept space.

System.out.println("Entername:");
System.out.println(name)

System.out.println("Enter Name 2:");

System.out.println(Name 2)

output: Entername
Input: FLM Edutech
output: FLM Edutech

Enter Name 2
FLM Edutech
FLM

(sc.nextInt)

* If we take input as integer first after that we cannot take string, it will skip that line so, we have to use sc.nextLine(); ~~(or) sc.next();~~

*[After usage of sc.nextInt we have to write one empty sc.nextLine ~~sc.next();~~ (or) ();]

*[Scanner is called resource.
At the ending we should close that resources.]

CONDITIONAL STATEMENTS (If & else)

* Yes & No statements.
* if else.

```
if (↓) {
    condition is true, execute if block other than it will execute
}           else block.
else {

}
```

Syntax:
```
if (condition) {
        → logic should be written in block.
}
```

Ex:
```
int a = 10
if (a > 0) {
    syso("Positive");
}
```

(or)
```
Scanner sc = new Scanner(System.in)
int a = sc.nextInt();
System.out.println("Enter the numbers:");
if (a > 0) {
    syso("Positive");
}
```

output: Enter the number:
5
positive

[-5
Empty output]

**

* If condition will work without else.
* else condition will not work with if().
* print() → print in same line.
  println() → print in different line.

Ex:

```
if (a>0) {
    syso("positive");
}
else if (a==0) {
    syso("zero")
}
else {
    syso("Negative")
}
```

→ if else ladder.

***

* Only if() → we can write.
* if() / else } we can write.

***

* if - else if - else - we can write.
* Only else - we can't write
* only else if - we can't write.
* only if - else if - we can write.

* Nested if():

→ write, if block inside if block.

```
if (condition) {
    if (condition2) {
    }
    else {
    }
}
```

Ex:

```
package com;
import java.util.Scanner;
public class NestedIf {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter marks :");
        int marks = sc.nextInt();
        if (marks > 50 && marks <=100) {
            System.out.println("Pass");
        }
        else {
            System.out.println("Fail");
        }
    }
}
```

⇒

```
if (marks > 50 && marks <=100) {
    System.out.println("pass");
    if (marks > 90 && marks <= 100) {
        System.out.println("Grade A");
    }
    else if (marks > 70 && marks <= 90) {
        System.out.println("Grade B");
    }
    else if (marks > 50 && marks <= 70) {
        System.out.println("Grade c");
    }
}
else {
    System.out.println("Failed");
}
```

Nested if

Output:

```
Enter marks :
75
pass
Grade B
```

Ex:

```
Scanner sc = new Scanner(System.in);
int age = sc.nextInt();
boolean hasLicense = true;
if (age >= 18) {
    if(hasLicense) { }
    System.out.println("You Can Drive");
    else {
        System.out.println("Apply for license");
    }
}
else {
    System.out.println("You are Minor you can not drive");
}
}
}
```

Output: Enter your age
19
You can Drive.

SWITCH STATEMENT

→ It will work as if else and if-else-Ladder.

* Syntax:

```
Switch (exp) {
    case exp1:
     statement;
      break;
    case exp2:
     Statement;
     break;
    case exp3;
      Statement;
      break;
    default:
       Statement;
```

* Boolean Values don't work in Switch Statement.
* Switch only designed to accept numbers, Character & String.

Ex:

```
package com;
import java.util.Scanner;
public class SwitchStatement {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();  → Syso("Enter Floor:");

    switch (num) {

    case 1 :
        System.out.println("1st floor");
    break;
    Case 2 :
        System.out.println("2nd floor");
    break;
    Case 3 :
        System.out.println("3rd Floor");
    break;
    dyault :
        System.out.println("Unknown Floor");

    }
}
}
```

output:

⇒ Enter Floor:
5
Unknown Floor

⇒ Enter Floor:
1
1st Floor
2nd floor
3rd floor
Unknown Floor.

output:

⇒ Enter Floor:
5
Unknown Floor

⇒ Enter Floor:
1
1st Floor
⇒ Enter Floor
2
2nd Floor.

*

ARGUMENTS

```
package com;
public class Arguments {
public static void main(String[] args) {
```

[Array - we can store multiple numbers | strings | Characters | boolean at a time]

array arguments.

Command Line Arguments

We can pass few arguments while running the Code. ⇒ Syso(args[0]) → Index.

// Array [10, 10, 20, 30, 50]

System.out.println(args[0]);
System.out.println(args[1]);
System.out.println(args[2]);

Class:7

21/04/2025

Run Configuration
↓
used to past argument!

LOOPS
(Repeated task).

Control + } to change font size.
Control − }

*1. FOR Loop
2. For Each Loop
3. While Loop
4. DO While Loop

* works only with bookeans.

* For Loop:

Ex:

```
package com;
public class practice {
    public static void main (String[] args) {
        syso (" Hello World");
        syso (" Hello world");
        syso (" Hello world");
        syso (" Hello world");
        syso (" Hello world");
    }
}
```

output: Hello world
Hello world
Hello world
Hello world
Hello world.

(8)

* For Loop: Used when you know how many times you have to do task. (If we know Range). It is also known as Entry Controlled Loops.

Syntax: for (↓) {}
1; 2; 3

It works with 3 things.
1. Initialization (Variable) [int a=0;]
2. Condition (a<5)
3. update the value.